Strong Indexability and Policy Gradient Methods

Deriving REINFORCE for Neural Whittle Index Learning

Saideekshith Vaddineni

IIIT Hyderabad

May 8, 2026

Definition 3: Strong Indexability

Definition 3: Strong Indexability

An arm is said to be **strongly indexable** if the discounted net reward $D_s(\lambda) = Q_{\lambda,\text{act}}(s) - Q_{\lambda,\text{pass}}(s)$ is **strictly decreasing** in λ for every state s.

Implications:

- There exists a unique $\lambda = W(s)$ such that $D_s(W(s)) = 0$.
- Monotonicity ensures that the optimal policy is a threshold policy.

Theorem 2: Whittle Accuracy

Theorem

If the arm is strongly indexable, then for any $\gamma > 0$ and an arbitrarily small positive constant δ, there exists $\epsilon > 0$ such that:

*If for any states s_0, s_1 and activation cost $\lambda \in [f_\theta(s_0) - \delta, f_\theta(s_0) + \delta]$, the reward of the neural network in $Env(\lambda)$ is at least $\max\{Q_{\lambda,act}(s_1), Q_{\lambda,pass}(s_1)\} - \epsilon$, then the network is γ-***Whittle-accurate***.*

Equivalent Statement for Proof: If the neural network is **not** γ-Whittle-accurate, then $\exists s_0, s_1$ and $\lambda \in [f_\theta(s_0) \pm \delta]$ such that the reward in $Env(\lambda)$ is strictly less than $\max\{Q_{\lambda,\text{act}}(s_1), Q_{\lambda,\text{pass}}(s_1)\} - \epsilon$.

Proof of Theorem 2: Contradiction

For a given γ, let:

$$\epsilon = \frac{1}{2} \min_s \{\min(Q_{W(s)+\gamma,\text{pass}}(s) - Q_{W(s)+\gamma,\text{act}}(s),\ Q_{W(s)-\gamma,\text{act}}(s) - Q_{W(s)-\gamma,\text{pass}}(s))\}$$

Case 1: $f_\theta(s) > W(s) + \gamma$ **(Overestimation)**

- Set $\lambda = f_\theta(s) - \delta$. Small δ ensures $\lambda > W(s) + \gamma$.
- **Optimal:** Passive. **Network:** Since $f_\theta(s) > \lambda$, it chooses **Active**.
- **Loss:** $Q_{\lambda,\text{pass}} - Q_{\lambda,\text{act}} \geq 2\epsilon \implies$ Reward $< \max\{Q\} - \epsilon$.

Case 2: $f_\theta(s) < W(s) - \gamma$ **(Underestimation)**

- Set $\lambda = f_\theta(s) + \delta$. Small δ ensures $\lambda < W(s) - \gamma$.
- **Optimal:** Active. **Network:** Since $f_\theta(s) < \lambda$, it chooses **Passive**.
- **Loss:** $Q_{\lambda,\text{act}} - Q_{\lambda,\text{pass}} \geq 2\epsilon \implies$ Reward $< \max\{Q\} - \epsilon$.

REINFORCE: Objective and Gradient

To measure policy quality, we define $J(\theta)$ as the expected return:

$$J(\theta) = V^{\pi}(s_0)$$

To maximize $J(\theta)$, we apply **Gradient Ascent** (opposite of the usual gradient descent in deep learning):

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} V^{\pi}(s_0)$$

Parameter Update Rule:

$$\theta_{t+1} = \theta_t + \alpha \cdot \nabla_{\theta} J(\theta)$$

REINFORCE: Policy Optimization

We estimate action quality using a **reward estimate** $\hat{Q}(s, a)$, which could be the Q-function, discounted return, or advantage function.

The update rule pushes the policy parameters towards actions with higher rewards:

$$\theta_{t+1} = \theta_t + \alpha \hat{Q}(s, a) \nabla \pi_{\theta_t}(a|s)$$

However, sampling bias (sampling high-probability actions more often) causes instability. To fix this, we normalize by the probability of the action:

$$\theta_{t+1} = \theta_t + \alpha \hat{Q}(s, a) \frac{\nabla \pi_{\theta_t}(a|s)}{\pi_\theta(a|s)}$$

REINFORCE: The Log-Derivative Trick

We use a basic fact from calculus to transform the update rule into a logarithmic form:

$$\nabla \log f(x) = \frac{\nabla f(x)}{f(x)}$$

Substituting this into our policy update equation, we arrive at the core of **REINFORCE**:

$$\theta_{t+1} = \theta_t + \alpha \hat{Q}(s, a) \nabla_\theta \log \pi_\theta(a|s)$$

This formulation is reminiscent of a logarithmic cross-entropy loss and ensures more stable updates by scaling gradients inversely with action probability.

NeurWIN: REINFORCE for Whittle Indices

NeurWIN treats the Whittle index $f_\theta(s)$ as the preference score.

NeurWIN Stochastic Policy: $\pi_\theta(a = 1|s, \lambda) = \sigma_m(f_\theta(s) - \lambda)$

Algorithm Sync:

1. **Sample Cost:** $\lambda \leftarrow f_\theta(s_0)$.
2. **Accumulate Gradients (h_e):** If $a = 1$, $h_e \leftarrow h_e + \nabla_\theta \ln(\sigma_m(f_\theta(s) - \lambda))$. If $a = 0$, use $(1 - \sigma_m)$.
3. **Baseline Normalization:** Calculate $\bar{G}_b$ (average reward).
4. **Final Update:** $\theta \leftarrow \theta + L_b(G_e - \bar{G}_b)h_e$.

H-SRAC: System Architecture Overview

The **Homogeneous State-Ranking Actor-Critic (H-SRAC)** algorithm consists of four synergistic components that bridge Deep RL with Whittle Index theory:

- **The Global Actor (π_θ):** A "Universal Ranker" that maps the entire state space to priority scores (Logits), ensuring homogeneous treatment across all arms.
- **The Newton-Balanced Sigmoid:** A differentiable "Budget Regulator" that shifts Actor scores using a global bias b to ensure exactly M arms are picked on average.
- **The State-Specific Critic (V_ϕ):** A "Value Estimator" that evaluates individual state quality to generate the Advantage (δ) signal for training.
- **Stochastic Sensitivity (λ):** A "Sharpness Control" sampled during training to force the Actor to learn robust rankings regardless of policy entropy.

1. Actor Forward Pass: Universal State Ranking

The Actor π_θ serves as a **Global Ranker**. It maps the constant state-space vector $\mathbf{S}_{all}$ to a priority logit vector $\mathbf{H}$:

$$\mathbf{H} = [h_1, h_2, \ldots, h_S] = \pi_\theta([s_1, s_2, \ldots, s_S])$$

State Retrieval:
For any physical arm i at time t, we retrieve the logit corresponding to its specific state $s(i)^t \in \mathcal{S}$:

$$h(s(i)^t) = \text{Lookup}(\mathbf{H}, s(i)^t)$$

Identical states across different machines receive identical priority scores.

2. Budget-Balanced Probability Mapping

We transform logits into selection probabilities $p_i^t \in [0, 1]$ while enforcing the budget M:

$$p_i^t = \sigma(\lambda h(s(i)^t) + b) = \frac{1}{1 + \exp(-(\lambda h(s(i)^t) + b))}$$

Solving for Global Bias b (Newton's Method):
We find the unique b such that the expected number of active arms is M:

$$f(b) = \sum_{i=1}^{N} p_i^t - M = 0$$

Iterative Update:

$$b_{next} = b - \frac{f(b)}{f'(b)} = b - \frac{\sum p_i^t - M}{\sum p_i^t(1 - p_i^t)}$$

3. H-SRAC: Proof of Expected Budget Constraint

We want to show that the expected number of active arms equals the budget M:

$$E\left[\sum_{i=1}^{N} a_i^t\right] = M$$

Proof:

1. By the **Linearity of Expectation**:

$$E\left[\sum_{i=1}^{N} a_i^t\right] = \sum_{i=1}^{N} E[a_i^t]$$

2. Since a_i^t is a Bernoulli random variable with probability p_i^t:

$$E[a_i^t] = P(a_i^t = 1) \cdot 1 + P(a_i^t = 0) \cdot 0 = p_i^t$$

3. Our Newton Solver specifically finds the bias b such that:

$$\sum_{i=1}^{N} p_i^t = \sum_{i=1}^{N} \sigma(\lambda h(s(i)^t) + b) = M$$

4. Substituting (2) and (3) into (1):

4. The Critic and Advantage Signal

The Critic V_ϕ estimates the value of a **single state**. We evaluate it for all N arms to compute the **Advantage Function** δ_i^t:

Temporal Difference (TD) Error:

$$\delta_i^t = \underbrace{r_i^t + \gamma V_\phi(s(i)^{t+1})}_{\text{Target}} - \underbrace{V_\phi(s(i)^t)}_{\text{Estimate}}$$

Interpretation:

- δ_i^t is the "error signal" indicating if an action was better than average.
- Large positive δ_i^t occurs when an arm avoids a deadline penalty F by being prioritized.

5. Dual-Network Parameter Updates

Critic Update (Value Refinement):
Minimize the Mean Squared Error of the TD-target:

$$\phi \leftarrow \phi + \alpha_\phi \sum_{i=1}^{N} \delta_i^t \nabla_\phi V_\phi(s(i)^t)$$

Actor Update (Ranking Refinement):
Update weights θ using the individual advantages via the **Straight-Through Estimator (STE)**:

$$\theta \leftarrow \theta + \alpha_\theta \sum_{i=1}^{N} \delta_i^t \nabla_\theta \log \pi_\theta(a_i^t \mid s(i)^t, \lambda)$$

$$\nabla_\theta \log \pi_\theta \approx \nabla_\theta \log \sigma(\lambda h(s(i)^t) + b)$$

6. Stochastic Sensitivity (λ) and Exploration

We sample $\lambda \sim \mathcal{U}(1, 20)$ per episode to vary policy sharpness.

The Exploration Trade-off:

- **Low λ (Soft):** Probabilities are flattened. This forces the network to explore the **Slack Zone** ($B < D$).
- **High λ (Hard):** Sigmoid approaches a step function. This simulates **Inference Time** (Top-M ranking).

This ensures the learned logits h_s converge to true **Whittle Index** values.

Problem 1: Wireless Scheduling Formulations

State $s[t]$**:** Vector $(y[t], v[t])$, where y is remaining load and $v \in \{0, 1\}$ is channel state.

Action $a[t]$**:** $a \in \{0, 1\}$.

Reward $r[t]$**:** $r[t] = \begin{cases} -\psi - \lambda & \text{if } a[t] = 1 \\ -\psi & \text{if } a[t] = 0 \end{cases}$

Next State $s[t+1]$**:**

$$s[t+1] = \begin{cases} (y[t] - r_2, 1) & \text{if } v[t] = 1, a[t] = 1 \\ (y[t] - r_1, 0) & \text{if } v[t] = 0, a[t] = 1 \\ (y[t], q(v[t])) & \text{otherwise} \end{cases}$$

Indexability: Proved in *Liu et al. (2015)*:
`https://dl.acm.org/doi/pdf/10.1145/2796314.2745851`.

Whittle Indices: $\hat{v}(y, 1) = \frac{cr_2}{y}$ and $v(y, 0) = \frac{c}{q(r_2/r_1)-1}$.

Problem 2: Deadline Scheduling Formulations

State $s[t]$**:** (D, B). D is time to deadline, B is job size.
Reward $r[t]$**:**

$$r[t] = \begin{cases} (1-c)a[t] & \text{if } B > 0, D > 1 \\ (1-c)a[t] - F(B - a[t]) & \text{if } B > 0, D = 1 \\ 0 & \text{otherwise} \end{cases}$$

where $F(x) = 0.2x^2$.
Next State $s[t+1]$**:** $D[t+1] = D[t] - 1$. If $D[t] \leq 1$, $s[t+1] = (D, B)$ with prob Q. Else, $B[t+1] = B[t] - a[t]$.

Indexability Proof: Deadline Scheduling

Theorem 3: The deadline scheduling problem is strongly indexable.
Proof Logic:

- **Scenario 1:** Policy *Act* and *Pass* merge at round 2.

$$D_s(\lambda) = 1 - c - \lambda + \beta^{D-1}(F(b+1) - F(b))$$

Linear in λ with negative slope $\implies$ strictly decreasing.

- **Scenario 2:** Policies diverge. *Pass* (with larger B) eventually activates at τ.

$$D_s(\lambda) = (1 - c - \lambda)(1 - \beta^{\tau-1})$$

Since $\beta < 1$, $(1 - \beta^{\tau-1}) > 0$, thus $D_s(\lambda)$ is strictly decreasing.

Whittle Index $v(D, B)$:

$$v(D,B) = \begin{cases} 1 - c & 1 \leq B \leq D-1 \\ \beta^{D-1}[F(B-D+1) - F(B-D)] + 1 - c & D \leq B \end{cases}$$

Problem 3: Recovering Bandits Formulations

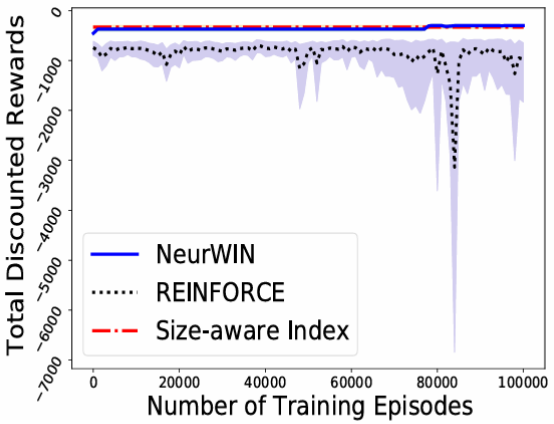
State $s[t]$**:** Waiting time $z[t] \in [1, z_{max}]$ since last play.
Action $a[t]$**:** $a \in \{0, 1\}$.

Reward $r[t]$**:** $r[t] = \begin{cases} f(z[t]) - \lambda & \text{if } a[t] = 1 \\ 0 & \text{if } a[t] = 0 \end{cases}$

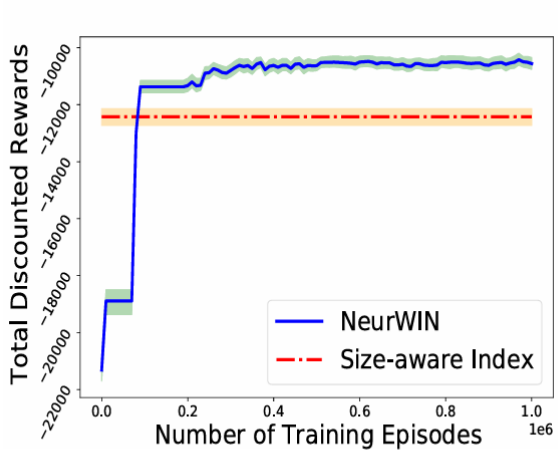
where $f(z) = \theta_0(1 - e^{-\theta_1 z})$.

Next State $s[t+1]$**:** $s[t+1] = \begin{cases} 1 & \text{if } a[t] = 1 \\ \min\{z[t] + 1, z_{max}\} & \text{if } a[t] = 0 \end{cases}$

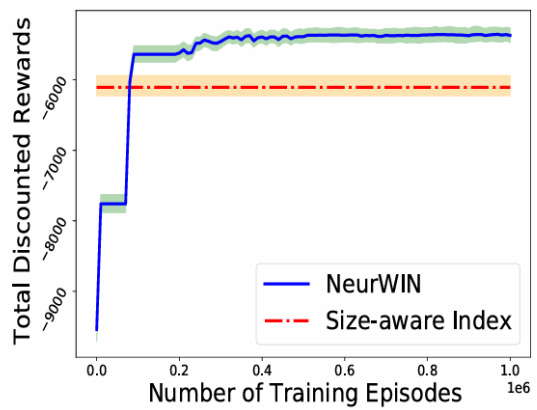
Problem 1: Wireless Scheduling Performance



(a) $N = 4.\ M = 1.$



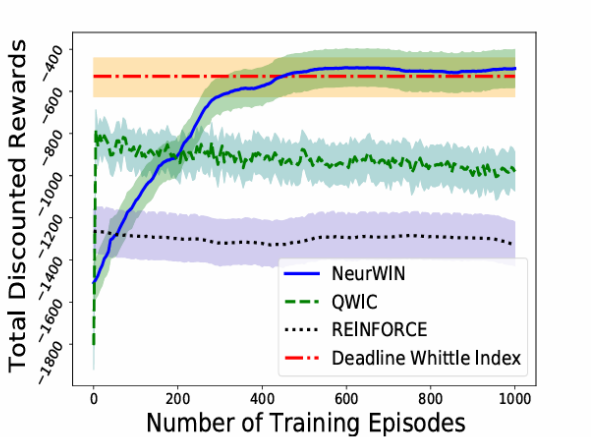
(b) $N = 100.\ M = 10.$



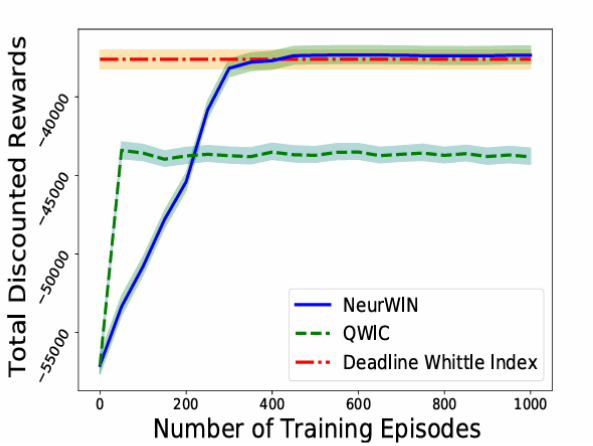
(c) $N = 100.\ M = 25.$

- Observation: NeurWIN closely tracks the optimal size-aware index.

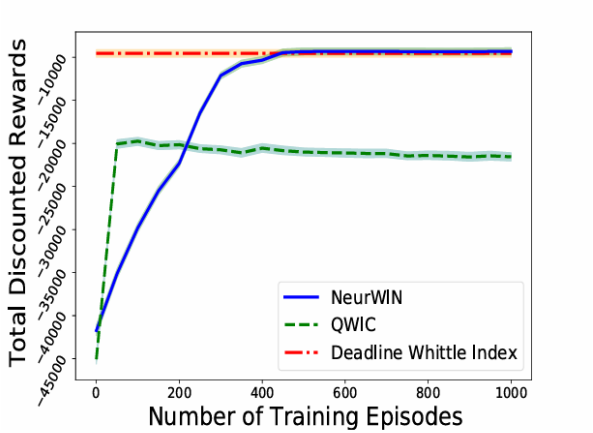
Problem 2: Deadline Scheduling Performance



(a) $N = 4$. $M = 1$.

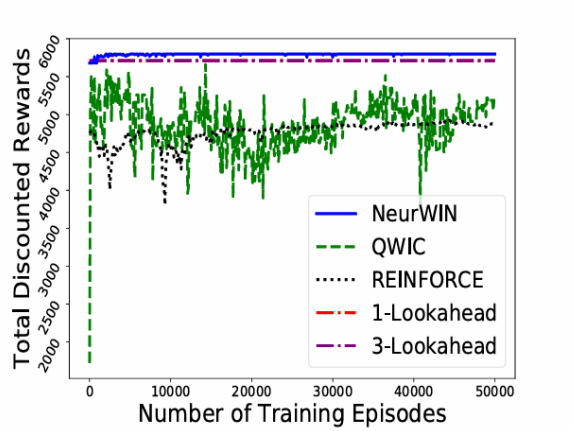


(b) $N = 100$. $M = 10$.

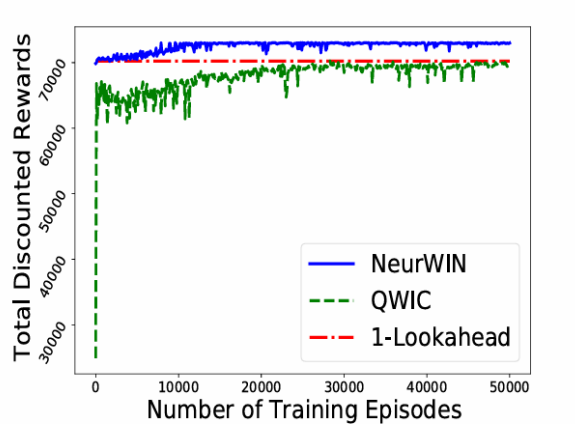


(c) $N = 100$. $M = 25$.

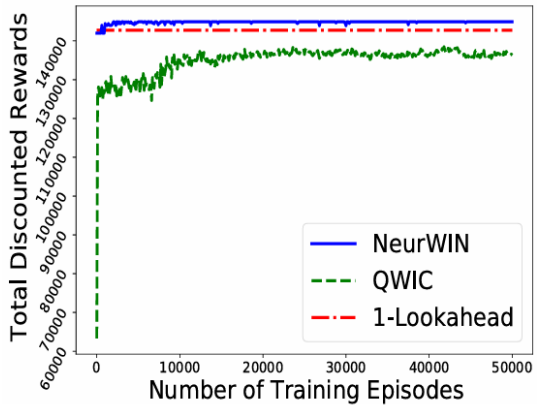
Problem 3: Recovering Bandits Performance



(a) $N = 4.\ M = 1.$



(b) $N = 100.\ M = 10.$



(c) $N = 100.\ M = 25.$